

EST-1141

Lenguajes de Programación

Juan Zamora O.

Orientación a Objetos



PONTIFICIA
UNIVERSIDAD
CATÓLICA DE
VALPARAÍSO

Estructura

1 Introducción

Introducción

- Inicios de OOP con **Simula** en los 60 en Noruega
 - Se incorpora la noción de objeto para representar objetos del mundo real
 - Inspirado en interacciones de objetos en el mundo real
- Programa es pensado como una colección de objetos interactivos independientes
- Durante los 80, el desarrollo de lenguajes en este paradigma explotó : Smalltalk, ADA, C++, Eiffel

Reutilización de SW e Independencia

- OOP satisface 3 necesidades importantes en el desarrollo de SW actual:
 - 1 Reutilización de componentes tanto como sea posible
 - 2 Modificación de programas haciendo la mínima cantidad de cambios en el código
 - 3 Mantener independencia entre componentes

Diseñar para reutilizar es el único objetivo de la OOP

- Se restringe el acceso a los detalles internos entre distintos componentes
 - ¿Conocen como está diseñado y codificado un `data.frame`?
- Modificaciones en algún componente solo impactan dentro de éste y no en la operación de otros componentes

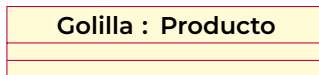
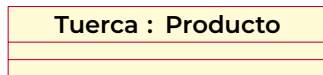
Clases y Objetos

- Un objeto es una cosa (tangible o intangible)
- Un programa orientado a objetos consiste de objetos que interactúan
- **Por ejemplo**, para controlar el inventario de los productos en una tienda, debieramos tener entre otros objetos a
 - Producto
 - Venta

- Un objeto está compuesto de datos y operaciones para manipular esos datos
- Por ejemplo, un objeto **Estudiante** puede
 - consistir de datos tales como: nombre, género, dirección.
 - tener operaciones para asignar y modificar estos valores.
- Retomando el ejemplo del inventario, algunos atributos y operaciones serían
 - Producto: código_barras, marca, precio, verificar_disponibilidad()
 - Venta: Producto, cantidad, descontar_venta()

Notación

- Para poder representar clases y objetos se utiliza (generalmente) la notación UML
- Cada objeto es representado como una caja con el nombre del objeto (instancia) y de la clase asociada o tipo asociada al objeto.



- Dentro de un programa escribimos instrucciones para crear objetos
- Para poder crear un objeto, es necesario entregar una definición llamada **clase**.
- Una **clase** es una especificación lo que pueden y no pueden hacer los objetos
- Un objeto es una instancia de una y solo una clase

Producto

Mensajes y Métodos

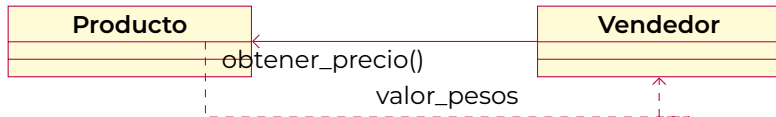
- Programas orientados a objetos son definidos sobre conjuntos de clases y, mientras corren, se usan esas clases y sus objetos creados para realizar tareas
- Una tarea puede ser cualquier operación,
 - sumar dos números
 - realizar cálculos en simulaciones de colisiones de objetos
 - controlar la IA en un juego de estrategia
- Para indicar a una clase o a un objeto que realice una determinada operación se le **envía un mensaje**

- Todos estos mensajes deben ser programados previamente al definir las clases
- Solo es posible enviar mensajes a las clases que **entienden** este mensaje
- **Entender** el mensaje quiere decir que puede ser procesado mediante un método definido en la clase
- Enviar un mensaje significa pedirle a un objeto que ejecute alguno de sus métodos

- En ocasiones este **traspaso** de mensajes es **bidireccional**
 - El objeto ejecuta la acción solicitada y retorna un valor (responde)



- En ocasiones este **traspaso** de mensajes es **bidireccional**
 - El objeto ejecuta la acción solicitada y retorna un valor (responde)



Valores de clase y de instancia

- Considere el ejemplo anterior del vendedor y el producto
- Se le indica a un objeto Producto que obtenga el valor en pesos ... ¿Donde se mantiene/almacena este valor?
 - En cada instancia, ya que esto depende del producto que cada vendedor consulte.
- Este es un valor de instancia
- Un valor de clase es compartido por todas las instancias.
 - Por ejemplo, todas las instancias de Vendedor comparten la misma dirección del local.

- Un **valor de clase** representa información compartida por todas las instancias o colectiva acerca de ellas
- Otra información relativa únicamente a una instancia en particular (`tipo_cliente` o `balance_actual`) se denomina **valor de instancia**
- Existen valores que pueden ser modificados y otros que no
 - Variables vs Constantes

Herencia

- Mecanismo usado para modelar dos o más entidades distintas, pero que **comparten** varios rasgos
- **Primero** se define una clase que contiene los rasgos comunes
- **Segundo** se definen clases que extienden la clase común,
 - **heredando** todo lo definido en esta última
 - incorporando además aquellos rasgos distintivos

- Permite modelar de manera más eficiente y clara (si se usa correctamente)

